

```

import { useState, useEffect, useRef } from "react";

// _____
// DESIGN TOKENS
// _____

const C = {
  dark:    "#1e1509",
  warm:    "#2e2318",
  clay:    "#8b5e3c",
  sand:    "#c9a87c",
  parch:   "#f5f0e8",
  cream:   "#faf8f3",
  sage:    "#5d6e52",
  gold:    "#c4922a",
  rust:    "#b85c38",
  mist:    "#e8f0f5",
  blue:    "#3d6a8a",
  white:   "#ffffff",
  ink:     "#1a1410",
  mid:     "#6a5848",
  light:   "#a89880",
};

// _____
// GLOBAL STYLES injected once
// _____

const GLOBAL_CSS = `
  @import url('https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght*,*::before,*::after { box-sizing: border-box; margin: 0; padding: 0; }
  body { font-family: 'DM Sans', sans-serif; background: ${C.cream}; color: ${C.ink}; }
  ::-webkit-scrollbar { width: 4px; }
  ::-webkit-scrollbar-track { background: transparent; }
  ::-webkit-scrollbar-thumb { background: ${C.sand}; border-radius: 2px; }
  button { cursor: pointer; border: none; background: none; font-family: inherit; }
  input, textarea { font-family: inherit; }
  textarea { resize: none; }
  @keyframes fadeUp { from { opacity:0; transform:translateY(12px); } to { opacity:1; transform:translateY(0); } }
  @keyframes pulse { 0%,100% { opacity:1; } 50% { opacity:0.4; } }
  @keyframes spin { to { transform: rotate(360deg); } }
  @keyframes bark { 0%,100%{transform:scale(1);} 20%{transform:scale(1.18);} 40%{transform:scale(1.18);} 60%{transform:scale(1.18);} 80%{transform:scale(1.18);} 100%{transform:scale(1);} }
  .fade-up { animation: fadeUp 0.4s ease both; }
  .spin { animation: spin 1s linear infinite; }
`;

```

```

function injectStyles() {
  if (document.getElementById("qc-styles")) return;
  const s = document.createElement("style");
  s.id = "qc-styles";
  s.textContent = GLOBAL_CSS;
  document.head.appendChild(s);
}

// _____
// AI CALL
// _____

async function callClaude(systemPrompt, userMessage, maxTokens = 900) {
  const res = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "claude-sonnet-4-20250514",
      max_tokens: maxTokens,
      system: systemPrompt,
      messages: [{ role: "user", content: userMessage }],
    }),
  });
  const data = await res.json();
  return data.content?.map(b => b.text || "").join("") || "";
}

// _____
// SHARED UI COMPONENTS
// _____

function NavBar({ screen, setScreen, dogName }) {
  const tabs = [
    { id: "home", icon: "🏠", label: "Home" },
    { id: "assess", icon: "🔍", label: "Assess" },
    { id: "log", icon: "📅", label: "Log" },
    { id: "train", icon: "🎓", label: "Train" },
    { id: "routine", icon: "📅", label: "Routine" },
  ];
  return (
    <nav style={{
      position: "fixed", bottom: 0, left: 0, right: 0,
      background: C.warm,
      display: "flex", borderTop: `1px solid rgba(201,168,124,0.15)`,
      zIndex: 100,
      boxShadow: "0 -4px 24px rgba(0,0,0,0.18)",
    }}>
      {tabs.map(t => {

```

```

const active = screen === t.id;
return (
  <button key={t.id} onClick={() => setScreen(t.id)} style={{
    flex: 1, padding: "10px 4px 8px",
    display: "flex", flexDirection: "column", alignItems: "center", gap:
    background: active ? "rgba(196,146,42,0.12)" : "transparent",
    borderTop: active ? `2px solid ${C.gold}` : "2px solid transparent",
    transition: "all 0.18s",
  }}>
    <span style={{ fontSize: 18, lineHeight: 1 }}>{t.icon}</span>
    <span style={{
      fontSize: 10, fontWeight: active ? 600 : 400,
      color: active ? C.sand : "rgba(201,168,124,0.45)",
      letterSpacing: "0.04em", textTransform: "uppercase",
    }}>{t.label}</span>
  </button>
);
}}}
</nav>
);
}

```

```

function Card({ children, style = {} }) {
  return (
    <div style={{
      background: C.white, borderRadius: 12,
      padding: "20px", marginBottom: 16,
      boxShadow: "0 2px 12px rgba(30,21,9,0.07)",
      border: `1px solid rgba(139,94,60,0.1)`,
      ...style,
    }}>
      {children}
    </div>
  );
}

```

```

function Btn({ children, onClick, variant = "primary", disabled = false, style =
const variants = {
  primary: { background: C.gold, color: C.dark, fontWeight: 700 },
  secondary: { background: "transparent", color: C.clay, border: `1.5px solid $
  dark: { background: C.warm, color: C.sand, fontWeight: 600 },
  ghost: { background: C.parch, color: C.clay, fontWeight: 500 },
  danger: { background: "#fdf0ec", color: C.rust, border: `1px solid rgba(184,9
};
return (
  <button onClick={onClick} disabled={disabled} style={{
    ...variants[variant],

```

```

        padding: "12px 20px", borderRadius: 8,
        fontSize: 14, letterSpacing: "0.02em",
        opacity: disabled ? 0.5 : 1,
        transition: "all 0.15s",
        display: "inline-flex", alignItems: "center", gap: 8,
        ...style,
    }}>
    {children}
</button>
);
}

function Tag({ children, color = C.clay }) {
    return (
        <span style={{
            display: "inline-block", fontSize: 11, fontWeight: 600,
            letterSpacing: "0.1em", textTransform: "uppercase",
            color: C.white, background: color,
            padding: "3px 10px", borderRadius: 20,
        }}>
            {children}
        </span>
    );
}

function Spinner() {
    return (
        <div style={{ display: "flex", flexDirection: "column", alignItems: "center",
            <div className="spin" style={{
                width: 32, height: 32, borderRadius: "50%",
                border: `3px solid ${C.parch}`, borderTopColor: C.gold,
            }} />
            <span style={{ fontSize: 13, color: C.light, fontStyle: "italic" }}>
                Thinking about your dog...
            </span>
        </div>
    );
}

function SectionTitle({ children }) {
    return (
        <h2 style={{
            fontFamily: "'Playfair Display', serif",
            fontSize: 22, fontWeight: 800, color: C.warm,
            marginBottom: 4, letterSpacing: "-0.01em",
        }}>
            {children}
        </h2>
    );
}

```

```

        </h2>
    );
}

function SubTitle({ children }) {
    return (
        <p style={{ fontSize: 14, color: C.mid, marginBottom: 20, lineHeight: 1.6 }}>
            {children}
        </p>
    );
}

// _____
// SCREEN: HOME / DASHBOARD
// _____

function HomeScreen({ dogName, setDogName, setScreen, logs, assessments }) {
    const [editing, setEditing] = useState(!dogName);
    const [nameInput, setNameInput] = useState(dogName || "");

    const save = () => {
        if (nameInput.trim()) { setDogName(nameInput.trim()); setEditing(false); }
    };

    const recentLog = logs[logs.length - 1];
    const recentAssess = assessments[assessments.length - 1];

    return (
        <div style={{ paddingBottom: 80 }}>
            { /* HERO */ }
            <div style={{
                background: C.warm, padding: "48px 24px 36px",
                position: "relative", overflow: "hidden",
            }}>
                <div style={{
                    position: "absolute", inset: 0,
                    background: "radial-gradient(ellipse 80% 60% at 50% 0%, rgba(196,146,42"
                }} />
                <div style={{ position: "relative", zIndex: 1 }}>
                    <div style={{ fontSize: 11, fontWeight: 600, letterSpacing: "0.2em", te
                        Your Dog Wellness Companion
                    </div>
                    <h1 style={{
                        fontFamily: "'Playfair Display', serif",
                        fontSize: 34, fontWeight: 800, color: "#f2ebe0",
                        lineHeight: 1.1, letterSpacing: "-0.02em", marginBottom: 8,
                    }}>
                        <em style={{ color: C.sand }}>Quiet</em> & Calm

```

```

    </h1>
    <p style={{ fontSize: 15, color: "rgba(242,235,224,0.55)", fontStyle: "
      Science-backed help for barking & anxiety
    </p>
  </div>
</div>

<div style={{ padding: "24px 20px 0" }}>

  { /* DOG NAME CARD */ }
  <Card style={{ background: C.parch, border: `1px solid rgba(196,146,42,0.
    { editing ? (
      <div>
        <div style={{ fontSize: 13, fontWeight: 600, color: C.clay, marginB
          What's your dog's name?
        </div>
        <div style={{ display: "flex", gap: 10 }}>
          <input
            value={nameInput}
            onChange={e => setNameInput(e.target.value)}
            onKeyDown={e => e.key === "Enter" && save()}
            placeholder="e.g. Buddy, Luna, Max..."
            style={{
              flex: 1, padding: "10px 14px", borderRadius: 8,
              border: `1.5px solid ${C.sand}`, background: C.white,
              fontSize: 16, color: C.ink, outline: "none",
            }}
            autoFocus
          />
          <Btn onClick={save}>Save</Btn>
        </div>
      </div>
    ) : (
      <div style={{ display: "flex", justifyContent: "space-between", align
        <div>
          <div style={{ fontSize: 12, color: C.light, textTransform: "upper
            <div style={{ fontSize: 22, fontFamily: "'Playfair Display', seri
              🐾 {dogName}
            </div>
          </div>
        </div>
        <Btn variant="ghost" onClick={() => setEditing(true)} style={{ padd
          Edit
        </Btn>
      </div>
    ) }
  </Card>

```

```

{ /* QUICK STATS */ }
<div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 12, m
  <Card style={{ margin: 0, textAlign: "center", padding: 16 }}>
    <div style={{ fontSize: 28, fontFamily: "'Playfair Display', serif",
      <div style={{ fontSize: 12, color: C.mid, fontWeight: 500, textTransf
    </Card>
  <Card style={{ margin: 0, textAlign: "center", padding: 16 }}>
    <div style={{ fontSize: 28, fontFamily: "'Playfair Display', serif",
      <div style={{ fontSize: 12, color: C.mid, fontWeight: 500, textTransf
    </Card>
  </div>

{ /* QUICK ACTIONS */ }
<Card>
  <div style={{ fontSize: 13, fontWeight: 600, color: C.clay, textTransfo
    Where would you like to start?
  </div>
  {[
    { icon: "🔍", label: "Get an AI Assessment", sub: "Tell me what's hap
    { icon: "📅", label: "Log a Barking Episode", sub: "Track patterns to
    { icon: "🎓", label: "Browse Training Protocols", sub: "Step-by-step
    { icon: "📅", label: "Build a Calming Routine", sub: "Create a daily
  ].map(a => (
    <button key={a.screen} onClick={() => setScreen(a.screen)} style={{
      width: "100%", display: "flex", alignItems: "center", gap: 14,
      padding: "14px 0", borderBottom: `1px solid ${C.parch}`,
      background: "none", textAlign: "left",
      transition: "opacity 0.15s",
    }}>
      <div style={{
        width: 44, height: 44, borderRadius: 10,
        background: `${a.color}18`,
        display: "flex", alignItems: "center", justifyContent: "center",
        fontSize: 20, flexShrink: 0,
      }}>
        {a.icon}
      </div>
      <div style={{ flex: 1, minWidth: 0 }}>
        <div style={{ fontWeight: 600, fontSize: 15, color: C.ink, margin
          <div style={{ fontSize: 13, color: C.mid, lineHeight: 1.4 }}>{a.s
        </div>
        <span style={{ color: C.light, fontSize: 18, flexShrink: 0 }}></sp
      </button>
    )})
  </Card>

{ /* RECENT */ }

```

```

    {recentAssess && (
      <Card style={{ background: "#fdf8f2", border: `1px solid rgba(196,146,4
        <div style={{ fontSize: 11, fontWeight: 700, color: C.gold, letterSpacing: 0.1em }}>
          Latest Assessment
        </div>
        <div style={{ fontSize: 15, fontWeight: 600, color: C.warm, marginBot
          <div style={{ fontSize: 13, color: C.mid, lineHeight: 1.6 }}>{recentA
            <div style={{ marginTop: 8, fontSize: 11, color: C.light }}>{recentAs
          </div>
        </Card>
      )}

    { /* SCIENCE FACT */ }
    <Card style={{ background: C.warm, border: "none" }}>
      <div style={{ fontSize: 10, fontWeight: 700, letterSpacing: "0.18em", t
        Did You Know?
      </div>
      <p style={{ fontSize: 14, color: "rgba(242,235,224,0.75)", lineHeight:
        "Up to 72% of dogs show anxiety-related behaviours at some point in t
      </p>
      <div style={{ fontSize: 11, color: "rgba(201,168,124,0.4)", marginTop:
    </Card>

    </div>
  </div>
);
}

// _____
// SCREEN: AI ASSESSMENT
// _____

const ASSESS_SYSTEM = `You are a certified canine behaviour expert and veterinary

```

Always respond in this EXACT JSON structure – no extra text outside the JSON:

```

{
  "barkType": "Short label e.g. Alert Barking, Separation Anxiety, Fear-Based Bar
  "confidence": "High / Medium / Low",
  "summary": "2-3 sentence plain-language summary of what is happening and why",
  "rootCause": "1-2 sentences on the underlying emotional/neurological cause",
  "immediateSteps": ["Step 1 (specific and actionable)", "Step 2", "Step 3"],
  "trainingApproach": "2-3 sentences on the best training strategy for this speci
  "redFlags": "Either null or a short string describing any signs that profession
  "encouragement": "One warm, honest sentence of encouragement for the owner"
}

```

Be specific to the dog and situation described. Use plain English. All scientific

```
function AssessScreen({ dogName, onSave }) {
```



```

const [step, setStep] = useState(0); // 0=form, 1=loading, 2=result
const [desc, setDesc] = useState("");
const [age, setAge] = useState("");
const [breed, setBreed] = useState("");
const [when, setWhen] = useState("");
const [result, setResult] = useState(null);
const [error, setError] = useState("");

const name = dogName || "your dog";

const whenOptions = [
  "When left alone", "At strangers / visitors", "At other dogs",
  "During storms / fireworks", "At all times / randomly", "When I'm home but in
];

const run = async () => {
  if (!desc.trim()) { setError("Please describe what's happening."); return; }
  setError("");
  setStep(1);
  const prompt = `Dog name: ${name}
Breed: ${breed || "not specified"}
Age: ${age || "not specified"}
Main trigger / when it happens: ${when || "not specified"}
Owner's description: ${desc}

Please assess this dog's barking and/or anxiety behaviour.`;
  try {
    const raw = await callClaude(ASSESS_SYSTEM, prompt, 800);
    const clean = raw.replace(/```json|```/g, "").trim();
    const parsed = JSON.parse(clean);
    setResult(parsed);
    onSave({ ...parsed, date: new Date().toLocaleDateString(), dog: name });
    setStep(2);
  } catch (e) {
    setError("Something went wrong. Please try again.");
    setStep(0);
  }
};

const reset = () => { setStep(0); setDesc(""); setResult(null); };

const confidenceColor = { High: C.sage, Medium: C.gold, Low: C.rust };

if (step === 1) return <div style={{ padding: "48px 20px" }}><Spinner /></div>;

if (step === 2 && result) return (
  <div style={{ padding: "24px 20px 100px" }} className="fade-up">

```

```

<SectionTitle>Assessment for {name}</SectionTitle>
<SubTitle>{result.date}</SubTitle>

{/* BARK TYPE */}
<Card style={{ background: C.warm, border: "none" }}>
  <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center" }}>
    <div style={{ fontSize: 11, fontWeight: 700, letterSpacing: "0.16em", textTransform: "uppercase" }}>
      Identified As
    </div>
    <Tag color={confidenceColor[result.confidence] || C.clay}>
      {result.confidence} Confidence
    </Tag>
  </div>
  <div style={{ fontFamily: "'Playfair Display', serif", fontSize: 24, fontStyle: "italic" }}>
    {result.barkType}
  </div>
  <p style={{ fontSize: 14, color: "rgba(242,235,224,0.7)", lineHeight: 1.7 }}>
    {result.summary}
  </p>
</Card>

{/* ROOT CAUSE */}
<Card style={{ background: "#f4faf4", border: `1px solid rgba(93,110,82,0.2)` }}>
  <div style={{ fontSize: 11, fontWeight: 700, letterSpacing: "0.14em", textTransform: "uppercase" }}>
    🐛 Root Cause
  </div>
  <p style={{ fontSize: 15, color: "#2a4a2a", lineHeight: 1.75 }}>{result.rootCause}
</Card>

{/* IMMEDIATE STEPS */}
<Card>
  <div style={{ fontSize: 13, fontWeight: 700, color: C.clay, textTransform: "uppercase" }}>
    ✅ Do These First
  </div>
  {result.immediateSteps?.map((s, i) => (
    <div key={i} style={{ display: "flex", gap: 12, marginBottom: 12, align-items: "center" }}>
      <div style={{
        width: 26, height: 26, borderRadius: "50%",
        background: C.gold, color: C.dark,
        fontWeight: 700, fontSize: 12,
        display: "flex", alignItems: "center", justifyContent: "center",
        flexShrink: 0, marginTop: 1,
      }}>{i + 1}</div>
      <p style={{ fontSize: 15, color: C.ink, lineHeight: 1.7, flex: 1 }}>{s}
    </div>
  ))}
</Card>

```

```

    { /* TRAINING APPROACH */ }
    <Card style={{ background: "#f0f5fa", border: `1px solid rgba(61,106,138,0.
      <div style={{ fontSize: 11, fontWeight: 700, letterSpacing: "0.14em", tex
        🎓 Training Approach
      </div>
      <p style={{ fontSize: 15, color: "#1a3a5a", lineHeight: 1.75 }}>{result.t
    </Card>

    { /* RED FLAGS */ }
    {result.redFlags && (
      <Card style={{ background: "#fdf0ec", border: `1px solid rgba(184,92,56,0
        <div style={{ fontSize: 11, fontWeight: 700, letterSpacing: "0.14em", t
          ⚠️ Worth Noting
        </div>
        <p style={{ fontSize: 15, color: "#5a2010", lineHeight: 1.7 }}>{result.
      </Card>
    )}

    { /* ENCOURAGEMENT */ }
    <Card style={{ background: C.parch, border: `1px solid rgba(196,146,42,0.2)
      <p style={{ fontSize: 16, fontFamily: "'Source Serif 4', serif", fontStyl
        🐾 "{result.encouragement}"
      </p>
    </Card>

    <div style={{ display: "flex", gap: 10 }}>
      <Btn onClick={reset} variant="secondary" style={{ flex: 1, justifyContent
        New Assessment
      </Btn>
    </div>
  </div>
);

return (
  <div style={{ padding: "24px 20px 100px" }}>
    <SectionTitle>AI Assessment</SectionTitle>
    <SubTitle>Describe what's happening with {name} and get a personalised anal

    {error && (
      <div style={{ background: "#fdf0ec", border: `1px solid ${C.rust}`, borde
        {error}
      </div>
    )}

    <Card>
      <label style={{ fontSize: 13, fontWeight: 600, color: C.clay, display: "b

```

```

    Breed (optional)
  </label>
  <input value={breed} onChange={e => setBreed(e.target.value)}
    placeholder="e.g. Labrador, Collie, Mixed breed..."
    style={{ width: "100%", padding: "10px 14px", borderRadius: 8, border:
  />

  <label style={{ fontSize: 13, fontWeight: 600, color: C.clay, display: "b
    Age (optional)
  </label>
  <input value={age} onChange={e => setAge(e.target.value)}
    placeholder="e.g. 3 years, 8 months..."
    style={{ width: "100%", padding: "10px 14px", borderRadius: 8, border:
  />

  <label style={{ fontSize: 13, fontWeight: 600, color: C.clay, display: "b
    When does it mainly happen?
  </label>
  <div style={{ display: "flex", flexWrap: "wrap", gap: 8, marginBottom: 16
    {whenOptions.map(o => (
      <button key={o} onClick={() => setWhen(when === o ? "" : o)} style={{
        padding: "7px 14px", borderRadius: 20, fontSize: 13, fontWeight: 50
        background: when === o ? C.gold : C.parch,
        color: when === o ? C.dark : C.mid,
        border: when === o ? "none" : `1px solid rgba(139,94,60,0.2)` ,
        transition: "all 0.15s",
      }}>
        {o}
      </button>
    ))}
  </div>

  <label style={{ fontSize: 13, fontWeight: 600, color: C.clay, display: "b
    Describe what's happening *
  </label>
  <textarea
    value={desc}
    onChange={e => setDesc(e.target.value)}
    placeholder={`Describe ${name}'s barking or anxiety in as much detail a
    rows={6}
    style={{
      width: "100%", padding: "12px 14px", borderRadius: 8,
      border: `1.5px solid ${desc ? C.sand : C.parch}`,
      fontSize: 15, color: C.ink, lineHeight: 1.7,
      outline: "none", background: C.cream,
      transition: "border-color 0.2s",
    }}
  </div>

```

```

    />
  </Card>

  <Btn onClick={run} disabled={!desc.trim()} style={{ width: "100%", justifyC
    🔍 Get My Assessment
  </Btn>
</div>
);
}

// _____
// SCREEN: BARK LOG
// _____
function LogScreen({ dogName, logs, setLogs }) {
  const [showForm, setShowForm] = useState(false);
  const [trigger, setTrigger] = useState("");
  const [duration, setDuration] = useState("");
  const [intensity, setIntensity] = useState(3);
  const [notes, setNotes] = useState("");
  const [animatingBtn, setAnimatingBtn] = useState(false);

  const name = dogName || "Your dog";

  const triggers = ["Doorbell / knock", "Stranger outside", "Other dog", "Alone /

  const save = () => {
    const entry = {
      id: Date.now(),
      trigger: trigger || "Not specified",
      duration: duration || "Unknown",
      intensity,
      notes,
      date: new Date().toLocaleDateString(),
      time: new Date().toLocaleTimeString([], { hour: "2-digit", minute: "2-digit
    };
    setLogs(prev => [...prev, entry]);
    setTrigger(""); setDuration(""); setIntensity(3); setNotes("");
    setShowForm(false);
  };

  const del = (id) => setLogs(prev => prev.filter(l => l.id !== id));

  const intensityColors = ["", C.sage, "#8bc48a", C.gold, "#e07a30", C.rust];
  const intensityLabels = ["", "Very Mild", "Mild", "Moderate", "Strong", "Severe

  // Simple frequency analysis
  const triggerCounts = logs.reduce((acc, l) => { acc[l.trigger] = (acc[l.trigger

```

```

const topTrigger = Object.entries(triggerCounts).sort((a, b) => b[1] - a[1])[0]

return (
  <div style={{ padding: "24px 20px 100px" }}>
    <SectionTitle>Bark Log</SectionTitle>
    <SubTitle>Track episodes to find patterns. Even a week of data reveals what

    { /* LOG BUTTON */ }
    <button
      onClick={() => { setAnimatingBtn(true); setTimeout(() => { setAnimatingBt
      style={{
        width: "100%", padding: "18px",
        background: C.warm, borderRadius: 12,
        border: `1px solid rgba(201,168,124,0.2)` ,
        marginBottom: 20, cursor: "pointer",
        animation: animatingBtn ? "bark 0.3s ease" : "none",
        display: "flex", alignItems: "center", justifyContent: "center", gap: 1
      }}
    >
      <span style={{ fontSize: 28 }}>🐶</span>
      <span style={{ fontFamily: "'Playfair Display', serif", fontSize: 20, fon
        Log a Barking Episode
      </span>
    </button>

    { /* LOG FORM */ }
    {showForm && (
      <Card className="fade-up">
        <div style={{ fontFamily: "'Playfair Display', serif", fontSize: 17, fo
          New Entry
        </div>

        <label style={{ fontSize: 13, fontWeight: 600, color: C.clay, display:
        <div style={{ display: "flex", flexWrap: "wrap", gap: 8, marginBottom:
          {triggers.map(t => (
            <button key={t} onClick={() => setTrigger(trigger === t ? "" : t)}
              padding: "6px 12px", borderRadius: 16, fontSize: 13,
              background: trigger === t ? C.gold : C.parch,
              color: trigger === t ? C.dark : C.mid,
              border: trigger === t ? "none" : `1px solid rgba(139,94,60,0.15)`
              transition: "all 0.15s",
            >{t}</button>
          )}}
        </div>

        <label style={{ fontSize: 13, fontWeight: 600, color: C.clay, display:
        <input value={duration} onChange={e => setDuration(e.target.value)}

```

```

        placeholder="e.g. 5 minutes, 30 seconds..."
        style={{ width: "100%", padding: "10px 14px", borderRadius: 8, border
/>

<label style={{ fontSize: 13, fontWeight: 600, color: C.clay, display:
  Intensity: <span style={{ color: intensityColors[intensity] }}>{inten
</label>
<div style={{ display: "flex", gap: 8, marginBottom: 16 }}>
  {[1, 2, 3, 4, 5].map(n => (
    <button key={n} onClick={() => setIntensity(n)} style={{
      flex: 1, height: 36, borderRadius: 6,
      background: intensity >= n ? intensityColors[n] : C.parch,
      border: "none", transition: "all 0.15s",
    }} />
  ))}
</div>

<label style={{ fontSize: 13, fontWeight: 600, color: C.clay, display:
<textarea value={notes} onChange={e => setNotes(e.target.value)}
  placeholder="What happened before, during, or after..."
  rows={3}
  style={{ width: "100%", padding: "10px 14px", borderRadius: 8, border
/>

<div style={{ display: "flex", gap: 10 }}>
  <Btn onClick={save} style={{ flex: 1, justifyContent: "center" }}>Sav
  <Btn onClick={() => setShowForm(false)} variant="ghost" style={{ padd
</div>
</Card>
))

{/* PATTERN INSIGHT */}
{logs.length >= 3 && topTrigger && (
  <Card style={{ background: "#fdf8f2", border: `1px solid rgba(196,146,42,
    <div style={{ fontSize: 11, fontWeight: 700, color: C.gold, letterSpacing: 0.5 }}>
       Pattern Detected
    </div>
    <p style={{ fontSize: 15, color: C.warm, lineHeight: 1.65 }}>
      <strong>{name}</strong>'s most common trigger</strong> is <strong style={{ color: C.warm }}>{trigger}</strong>.
      This pattern helps target your training more precisely.
    </p>
  </Card>
)}

{/* LOG ENTRIES */}
{logs.length === 0 ? (
  <Card style={{ textAlign: "center", padding: "36px 20px", background: C.p

```

```

    <div style={{ fontSize: 32, marginBottom: 12 }}>📅</div>
    <p style={{ color: C.mid, fontSize: 15 }}>No episodes logged yet.<br />
  </Card>
) : (
  [...logs].reverse().map(l => (
    <Card key={l.id} style={{ padding: "16px 18px" }}>
      <div style={{ display: "flex", justifyContent: "space-between", align
        <div style={{ flex: 1 }}>
          <div style={{ display: "flex", gap: 8, alignItems: "center", margi
            <Tag color={intensityColors[l.intensity]}>{intensityLabels[l.in
              <span style={{ fontSize: 12, color: C.light }}>{l.date} · {l.ti
            </div>
          <div style={{ fontSize: 15, fontWeight: 600, color: C.warm, margi
            {l.duration} && <div style={{ fontSize: 13, color: C.mid }}>Durati
            {l.notes} && <div style={{ fontSize: 13, color: C.mid, marginTop:
          </div>
          <button onClick={() => del(l.id)} style={{ color: "#ccc", fontSize:
        </div>
      </Card>
    ))
  )}
</div>
);
}

// _____
// SCREEN: TRAINING PROTOCOLS
// _____

const PROTOCOLS = [
  {
    id: "quiet",
    icon: "😬",
    title: "The 'Quiet' Command",
    tag: "Barking",
    tagColor: C.rust,
    summary: "Teach a reliable verbal cue to stop barking on command – one of the
    steps: [
      { title: "Prepare your setup", body: "Gather high-value treats (pea-sized p
      { title: "Trigger, then interrupt", body: "Trigger the bark intentionally.
      { title: "Mark and reward the silence", body: "The instant your dog goes qu
      { title: "Extend the duration gradually", body: "Over multiple sessions, re
      { title: "Fade the treat lure", body: "Once your dog quiets reliably on cue
      { title: "Generalise the command", body: "Practise in different rooms, at d
    ],
  },
  {
    id: "sa",

```



```

    icon: "👤",
    title: "Graduated Departure Protocol",
    tag: "Separation Anxiety",
    tagColor: C.blue,
    summary: "The evidence-based treatment for dogs that bark, howl, or panic whe
    steps: [
      { title: "Find the anxiety threshold", body: "Before anything else, find ou
      { title: "Practise departure cues without departing", body: "Pick up your k
      { title: "Begin micro-absences", body: "Step outside for 5 seconds. Return
      { title: "Introduce the departure anchor", body: "Give a high-value, long-l
      { title: "Extend duration in small increments", body: "Progress slowly: 30
      { title: "Never return to a distressed dog", body: "If your dog has been di
    ],
  },
  {
    id: "dscc",
    icon: "🧠",
    title: "Desensitisation & Counter-Conditioning",
    tag: "Anxiety & Fear",
    tagColor: C.sage,
    summary: "The most powerful technique in animal behaviour for changing how a
    steps: [
      { title: "Identify the trigger and the threshold distance", body: "Find the
      { title: "Pair trigger appearance with high-value food", body: "The moment
      { title: "Watch for the emotional shift", body: "You are watching for the m
      { title: "Only reduce distance when completely calm", body: "After multiple
      { title: "Keep sessions short and positive", body: "5-10 minutes maximum. A
      { title: "Build gradually to real-world situations", body: "Eventually you
    ],
  },
  {
    id: "enrichment",
    icon: "🌱",
    title: "Enrichment for Calm",
    tag: "Anxiety Reduction",
    tagColor: C.clay,
    summary: "Mental stimulation reduces cortisol, lowers baseline anxiety, and g
    steps: [
      { title: "Replace the food bowl with foraging", body: "Instead of feeding f
      { title: "Introduce daily sniff walks", body: "On at least one walk per day
      { title: "Try nose work and scent games", body: "Hide treats around the roo
      { title: "Add short training sessions daily", body: "Two 5-minute training
      { title: "Rotate and vary the enrichment", body: "Dogs habituate to the sam
    ],
  },
];

```

```

function TrainScreen() {
  const [selected, setSelected] = useState(null);
  const [activeStep, setActiveStep] = useState(0);
  const [completedSteps, setCompletedSteps] = useState({});

  const toggleStep = (protocolId, stepIdx) => {
    const key = `${protocolId}-${stepIdx}`;
    setCompletedSteps(prev => ({ ...prev, [key]: !prev[key] }));
  };

  if (selected) {
    const p = PROTOCOLS.find(x => x.id === selected);
    const doneCount = p.steps.filter((_, i) => completedSteps[`${p.id}-${i}`]).length
    return (
      <div style={{ padding: "24px 20px 100px" }} className="fade-up">
        <button onClick={() => setSelected(null)} style={{ color: C.clay, fontSiz
          ← Back to Protocols
        </button>

        <div style={{ display: "flex", gap: 10, alignItems: "flex-start", marginB
          <span style={{ fontSize: 32 }}>{p.icon}</span>
          <div>
            <Tag color={p.tagColor}>{p.tag}</Tag>
            <h2 style={{ fontFamily: "'Playfair Display', serif", fontSize: 22, f
          </div>
        </div>
        <p style={{ fontSize: 15, color: C.mid, lineHeight: 1.7, marginBottom: 20

        /* PROGRESS */
        <div style={{ background: C.parch, borderRadius: 8, padding: "12px 16px",
          <div style={{ display: "flex", justifyContent: "space-between", fontSiz
            <span>Progress</span>
            <span>{doneCount} / {p.steps.length} steps</span>
          </div>
          <div style={{ height: 6, background: "rgba(139,94,60,0.15)", borderRadi
            <div style={{ height: "100%", width: `${(doneCount / p.steps.length)
          </div>
        </div>

        /* STEPS */
        {p.steps.map((s, i) => {
          const done = !!completedSteps[`${p.id}-${i}`];
          const open = activeStep === i;
          return (
            <Card key={i} style={{ padding: 0, overflow: "hidden", border: done ?
              <button onClick={() => setActiveStep(open ? -1 : i)} style={{
                width: "100%", padding: "16px 18px",

```

```

        display: "flex", alignItems: "center", gap: 14,
        background: "none", textAlign: "left",
      }}>
      <div style={{
        width: 30, height: 30, borderRadius: "50%", flexShrink: 0,
        background: done ? C.sage : open ? C.gold : C.parch,
        display: "flex", alignItems: "center", justifyContent: "center"
        fontSize: 13, fontWeight: 700,
        color: done || open ? C.white : C.mid,
        transition: "all 0.2s",
      }}>
        {done ? "✓" : i + 1}
      </div>
      <span style={{ flex: 1, fontWeight: 600, fontSize: 15, color: don
      <span style={{ color: C.light, fontSize: 16 }}>{open ? "▼" : ">"
    </button>
    {open && (
      <div style={{ padding: "0 18px 18px 18px" }} className="fade-up">
        <p style={{ fontSize: 15, color: C.ink, lineHeight: 1.8, margin
        <button onClick={() => toggleStep(p.id, i)} style={{
          padding: "9px 18px", borderRadius: 8, fontSize: 13, fontWeigh
          background: done ? "#fdf0ec" : C.sage,
          color: done ? C.rust : C.white,
          border: "none", transition: "all 0.15s",
        }}>
          {done ? "Mark Incomplete" : "Mark Complete ✓"}
        </button>
      </div>
    )}
  </Card>
);
  )}
</div>
);
}

return (
  <div style={{ padding: "24px 20px 100px" }}>
    <SectionTitle>Training Protocols</SectionTitle>
    <SubTitle>Evidence-based, step-by-step guides for every type of barking and

    {PROTOCOLS.map(p => (
      <button key={p.id} onClick={() => { setSelected(p.id); setActiveStep(0);
        width: "100%", textAlign: "left", background: C.white,
        borderRadius: 12, padding: "20px",
        border: `1px solid rgba(139,94,60,0.1)`,
        boxShadow: "0 2px 12px rgba(30,21,9,0.07)",

```

```

        marginBottom: 14, transition: "all 0.18s",
        display: "block",
    }}>
    <div style={{ display: "flex", gap: 14, alignItems: "flex-start" }}>
        <div style={{
            width: 52, height: 52, borderRadius: 12, flexShrink: 0,
            background: `${p.tagColor}18`,
            display: "flex", alignItems: "center", justifyContent: "center",
            fontSize: 24,
        }}>
            {p.icon}
        </div>
        <div style={{ flex: 1, minWidth: 0 }}>
            <div style={{ marginBottom: 6 }}><Tag color={p.tagColor}>{p.tag}</T
            <div style={{ fontFamily: "'Playfair Display', serif", fontSize: 18
            <p style={{ fontSize: 14, color: C.mid, lineHeight: 1.6 }}>{p.summa
            <div style={{ fontSize: 13, color: C.light, marginTop: 8, fontWeigh
            </div>
            </div>
        </button>
    )})
</div>
);
}

// _____
// SCREEN: DAILY ROUTINE BUILDER
// _____

const ROUTINE_ITEMS = [
    { id: "morningwalk", time: "Morning", icon: "🌅", label: "Morning sniff walk"
    { id: "breakfast", time: "Morning", icon: "🍲", label: "Foraging breakfast"
    { id: "training1", time: "Morning", icon: "🎓", label: "Training session 1"
    { id: "safespace", time: "Morning", icon: "🛏", label: "Safe space rest per
    { id: "departure", time: "Midday", icon: "🚪", label: "Calm departure rout
    { id: "enrichment", time: "Midday", icon: "🌿", label: "Enrichment activity
    { id: "afternoon", time: "Afternoon", icon: "🐕", label: "Afternoon walk",
    { id: "training2", time: "Afternoon", icon: "🎓", label: "Training session 2"
    { id: "lickmat", time: "Evening", icon: "😋", label: "Evening lick mat",
    { id: "winddown", time: "Evening", icon: "🌙", label: "Evening wind-down",
];

const categoryColors = { exercise: C.sage, enrichment: C.gold, training: C.blue,

function RoutineScreen({ dogName }) {
    const [selected, setSelected] = useState(new Set(["morningwalk", "breakfast", "
    const [generated, setGenerated] = useState(false);
    const [aiTip, setAiTip] = useState("");

```

```

const [loadingTip, setLoadingTip] = useState(false);

const name = dogName || "your dog";
const toggle = (id) => setSelected(prev => { const n = new Set(prev); n.has(id)

const times = ["Morning", "Midday", "Afternoon", "Evening"];

const getTip = async () => {
  setLoadingTip(true);
  const selectedItems = ROUTINE_ITEMS.filter(i => selected.has(i.id)).map(i =>
  const tip = await callClaude(
    "You are a concise canine behaviour expert. Give a 2-3 sentence practical t
    `The owner has built this routine for ${name}: ${selectedItems}. Give one k
    200
  );
  setAiTip(tip);
  setLoadingTip(false);
  setGenerated(true);
};

const completedByTime = (time) => ROUTINE_ITEMS.filter(i => i.time === time &&

return (
  <div style={{ padding: "24px 20px 100px" }}>
    <SectionTitle>Daily Routine Builder</SectionTitle>
    <SubTitle>Predictability is one of the most powerful anti-anxiety tools ava

    {/* SELECTOR */}
    {times.map(time => (
      <div key={time} style={{ marginBottom: 24 }}>
        <div style={{ fontSize: 11, fontWeight: 700, letterSpacing: "0.16em", t
          {time}
        </div>
        {ROUTINE_ITEMS.filter(i => i.time === time).map(item => {
          const active = selected.has(item.id);
          return (
            <button key={item.id} onClick={() => toggle(item.id)} style={{
              width: "100%", textAlign: "left",
              background: active ? `${categoryColors[item.category]}12` : C.whi
              borderRadius: 10, padding: "14px 16px", marginBottom: 8,
              border: active ? `1.5px solid ${categoryColors[item.category]}` :
              display: "flex", gap: 14, alignItems: "flex-start",
              transition: "all 0.18s",
              boxShadow: active ? `0 2px 10px ${categoryColors[item.category]}2
            >>
            <div style={{
              width: 40, height: 40, borderRadius: 9, flexShrink: 0,

```

```

        background: active ? `${categoryColors[item.category]}22` : C.p
        display: "flex", alignItems: "center", justifyContent: "center"
        fontSize: 18, transition: "all 0.18s",
    }}>{item.icon}</div>
<div style={{ flex: 1 }}>
    <div style={{ fontWeight: 600, fontSize: 15, color: active ? ca
        {item.label}
    </div>
    <div style={{ fontSize: 13, color: C.mid, lineHeight: 1.5 }}>{i
</div>
<div style={{
    width: 22, height: 22, borderRadius: "50%", flexShrink: 0,
    background: active ? categoryColors[item.category] : "transpare
    border: `2px solid ${active ? categoryColors[item.category] : "
    display: "flex", alignItems: "center", justifyContent: "center"
    color: C.white, fontSize: 12, fontWeight: 700,
    transition: "all 0.18s",
    >
    {active && "✓"}
</div>
</button>

    );
    }}}
</div>
)}}

{/* SUMMARY */}
<Card style={{ background: C.warm, border: "none" }}>
    <div style={{ fontSize: 11, fontWeight: 700, color: "rgba(201,168,124,0.5
        {name}'s Routine - {selected.size} activities selected
    </div>
    {times.map(t => {
        const items = completedByTime(t);
        if (!items.length) return null;
        return (
            <div key={t} style={{ marginBottom: 12 }}>
                <div style={{ fontSize: 12, color: C.sand, fontWeight: 600, textTra
                    {items.map(i => (
                        <div key={i.id} style={{ display: "flex", gap: 8, alignItems: "ce
                            <span style={{ fontSize: 14 }}>{i.icon}</span>
                            <span style={{ fontSize: 14, color: "rgba(242,235,224,0.75)" }}
                        </div>
                    ))}
                </div>
            )}}
        </div>
    );
    }}}
    {selected.size === 0 && <p style={{ color: "rgba(242,235,224,0.4)", fonts

```

```

</Card>

{/* AI TIP */}
{selected.size >= 3 && (
  <div>
    <Btn onClick={getTip} disabled={loadingTip} style={{ width: "100%", jus
      {loadingTip ? "Getting tip..." : "✨ Get AI Implementation Tip"}
    </Btn>
    {loadingTip && <Spinner />}
    {generated && aiTip && (
      <Card style={{ background: "#f4faf4", border: `1px solid rgba(93,110,
        <div style={{ fontSize: 11, fontWeight: 700, color: C.sage, lettersS
          🌿 Tip for Your Routine
        </div>
        <p style={{ fontSize: 15, color: "#2a4a2a", lineHeight: 1.8, fontFa
      </Card>
    )}
  </div>
)}
</div>
);
}

// _____
// ROOT APP
// _____

export default function App() {
  injectStyles();

  const [screen, setScreen] = useState("home");
  const [dogName, setDogName] = useState("");
  const [logs, setLogs] = useState([]);
  const [assessments, setAssessments] = useState([]);

  const screenMap = {
    home: <HomeScreen dogName={dogName} setDogName={setDogName} setScreen={set
    assess: <AssessScreen dogName={dogName} onSave={a => setAssessments(prev =>
    log: <LogScreen dogName={dogName} logs={logs} setLogs={setLogs} />,
    train: <TrainScreen />,
    routine: <RoutineScreen dogName={dogName} />,
  };

  return (
    <div style={{ maxWidth: 480, margin: "0 auto", minHeight: "100vh", background
      <div key={screen} className="fade-up">
        {screenMap[screen]}
      </div>
    </div>
  )
}

```

```
        <NavBar screen={screen} setScreen={setScreen} dogName={dogName} />
    </div>
  );
}
```